

Name: Dhruv Makwana

Module 16) Python DB and Framework

1.1) Introduction to embedding HTML within Python using web frameworks like Django or Flask.

Ans: Frameworks like Django and Flask allow developers to generate HTML dynamically using Python.

Instead of static pages, data is passed from Python code to HTML templates.

This enables dynamic websites like dashboards, blogs, etc.

1.2) Generating dynamic HTML content using Django templates.

Ans: Django templates use variables and tags like `{{ }}` and `{% %}`.

Data from views is passed to templates and rendered dynamically.

Example: `{{ name }}` displays user data on the page.

1.3) Integrating CSS with Django templates.

Ans: CSS is linked using `{% load static %}` in templates.

Then use `<link rel="stylesheet" href="{% static 'style.css' %}">`.

This helps in styling web pages consistently.

1.4) How to serve static files (like CSS, JavaScript) in Django.

Ans: Define `STATIC_URL` and `STATICFILES_DIRS` in `settings.py`.

Store files in a static folder.

Use `{% static %}` tag to access them in templates.

1.5) Using JavaScript for client-side interactivity in Django templates.

Ans: JavaScript runs in the browser to handle actions like clicks and form validation.

It improves user experience without reloading pages.

Django templates can include JS code easily.

1.6) Linking external or internal JavaScript files in Django.

Ans: Internal JS: `<script src="{% static 'app.js' %}"></script>`

External JS: `<script src="https://..."></script>`

Use `{% load static %}` for local files.

1.7) Overview of Django: Web development framework.

Ans: Django is a high-level Python framework used for rapid web development.

It follows MVT architecture and provides built-in tools.

Used for secure and scalable applications.

1.8) Advantages of Django (e.g., scalability, security).

Ans: Django provides built-in security (CSRF, SQL injection protection).

Highly scalable and reusable components.

Fast development with built-in admin panel.

1.9) Django vs. Flask comparison: Which to choose and why.

Ans: Django is full-featured and best for large projects.

Flask is lightweight and flexible for small apps.

Choose Django for scalability, Flask for simplicity.

1.10) Understanding the importance of a virtual environment in Python projects.

Ans: Virtual environments isolate project dependencies.

Avoid conflicts between different project libraries.

Helps maintain clean and organized development.

1.11) Using venv or virtualenv to create isolated environments.

Ans: Use `python -m venv env` to create environment.

Activate using `env\Scripts\activate` (Windows).

Install packages without affecting system Python.

1.12) Steps to create a Django project and individual apps within the project.

Ans: Install Django → `django-admin startproject projectname`.

Create app → `python manage.py startapp appname`.

Add app in `INSTALLED_APPS`.

1.13) Understanding the role of `manage.py`, `urls.py`, and `views.py`.

Ans: `manage.py` runs commands like server and migrations.

`urls.py` handles routing of URLs.

`views.py` contains logic to process requests.

1.14) Django's MVT (Model-View-Template) architecture and how it handles request-response cycles.

Ans: Model handles database, View handles logic, Template handles UI.

Request → URL → View → Template → Response.

Ensures separation of concerns.

1.15) Introduction to Django's built-in admin panel.

Ans: Django provides a ready-to-use admin interface.

Used to manage database records easily.

Accessible via `/admin` route.

1.16) Customizing the Django admin interface to manage database records.

Ans: Register models in `admin.py`.

Customize list display, filters, search options.

Enhances admin usability.

1.17) Setting up URL patterns in `urls.py` for routing requests to views.

Ans: Define URL paths using `path()` function.

Map URLs to views.

Example: `path('home/', views.home)`.

1.18) Integrating templates with views to render dynamic HTML content.

Ans: Use `render(request, 'file.html', data)` in views.

Pass context data to templates.

Templates display dynamic content.

1.19) Using JavaScript for front-end form validation.

Ans: JavaScript checks user input before submission.

Improves performance and reduces server load.

Example: email validation.

1.20) Connecting Django to a database (SQLite or MySQL).

Ans: Configure database in `settings.py`.

SQLite is default, MySQL needs setup.

Run migrations to create tables.

1.21) Using the Django ORM for database queries.

Ans: ORM allows database operations using Python code.

No need to write SQL queries.

Example: `Model.objects.all()`.

1.22) Understanding Django's ORM and how QuerySets are used to interact with the database.

Ans: QuerySets represent database queries.

They are lazy and efficient.

Used for filtering and retrieving data.

1.23) Using Django's built-in form handling.

Ans: Django provides `forms.py` for form creation.

Handles validation and data processing.

Simplifies form handling.

1.24) Implementing Django's authentication system (sign up, login, logout, password management).

Ans: Django provides login, logout, signup features.

Uses built-in User model.

Supports password hashing and security.

1.25) Using AJAX for making asynchronous requests to the server without reloading the page.

Ans: AJAX allows data fetching without reloading page.

Uses JavaScript and APIs.

Improves user experience.

1.26) Techniques for customizing the Django admin panel.

Ans: Use ModelAdmin class for customization.

Add filters, search fields, inline models.

Enhances admin control.

1.27) Introduction to integrating payment gateways (like Paytm) in Django projects.

Ans: Use APIs provided by Paytm.

Handle payment requests and responses securely.

Requires merchant account and keys.

1.28) Steps to push a Django project to GitHub.

Ans: Initialize Git → git init.

Add files → git add . → git commit.

Push to GitHub repository.

1.29) Introduction to deploying Django projects to live servers like PythonAnywhere.

Ans: Upload code to PythonAnywhere.

Configure WSGI file and virtual environment.

Set static files and database.

1.30) Setting up social login options (Google, Facebook, GitHub) in Django using OAuth2.

Ans: Use OAuth2 authentication with libraries like Django -all auth.

Register app with providers like Google.

Get client ID and secret keys.

1.31) Integrating Google Maps API into Django projects.

Ans: Use API key from Google Maps API.

Embed maps using JavaScript in templates.

Display location data dynamically.